PROJECT REPORT

AGENTIC SOC: AN AUTONOMOUS LEVEL-1 SECURITY ANALYST USING ENSEMBLE MACHINE LEARNING, OSINT THREAT INTELLIGENCE AND LLM-BASED REASONING

Submitted by

THARUN S V

in partial fulfilment of the requirements for the degree of

MASTER OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

II Year

November 2026

TABLE OF CONTENTS

1. TITLE.....2
2. ABSTRACT.....3
3. BASE PAPER.....4
3.1 Summary.....4
3.2 Methodology.....4
3.3 Limitations Addressed by This Project.....4
3.4 Extension in This Project.....4
4. LITERATURE SURVEY.....5
4.1 Research Gap.....7
5. PROJECT REPORT.....8
5.1 Introduction.....8
5.2 Problem Statement.....8
5.3 Objectives.....8
5.4 Existing System vs Proposed System.....8
5.5 System Architecture.....9
5.6 Modules.....10
5.7 System Requirements.....10
5.8 Methodology.....10
6. IMPLEMENTATION.....12
6.1 Data Preparation and Feature Engineering.....12
6.2 Model Training.....12
6.3 OSINT Threat Intelligence.....13
6.4 Detection API.....13
6.5 LLM Reasoning.....13
6.6 Automated Response (SOAR) and Sandbox Execution.....14
6.7 Blockchain Audit Log.....14
6.8 Persistence and Authentication.....14
6.9 Dashboard.....14
6.10 Feedback and Retraining.....16
7. TESTING.....17
8. RESULTS AND DISCUSSION.....18
8.1 Detection Performance.....18
8.2 Per-Attack-Type Detection.....19
8.3 Baseline Comparison.....19
8.4 End-to-End Latency.....20
8.5 Discussion.....20
9. CONCLUSION AND FUTURE WORK.....22
9.1 Conclusion.....22
9.2 Future Work.....22
REFERENCES.....23
APPENDIX A: SAMPLE LLM DECISION FORMAT.....25
APPENDIX B: SAMPLE ISOLATION PLAYBOOK.....26
APPENDIX C: USER MANUAL.....27

1. TITLE

AGENTIC SOC: AN AUTONOMOUS LEVEL-1 SECURITY ANALYST USING ENSEMBLE MACHINE LEARNING, OSINT THREAT INTELLIGENCE AND LLM-BASED REASONING

Domain: Cyber Security / Machine Learning

Student: THARUN S V, M.E. Computer Science and Engineering, II Year

Project Type: Full project (Phase 1 + Phase 2)

2. ABSTRACT

Security Operations Centres (SOCs) receive thousands of alerts a day, and Level-1 analysts spend hours triaging each one by hand. Signature-based tools miss new and disguised attacks, while pure machine-learning detectors raise alerts without explaining them. Both slow down the Mean Time to Detect (MTTD) and the Mean Time to Respond (MTTR).

This project builds an Agentic SOC that automates the full Level-1 workflow: detect, explain, decide, contain and record. Incoming network and authentication logs first pass an OSINT stage that matches IPs, domains, ports and file hashes against a local indicator list and live reputation feeds (AbuseIPDB and AlienVault OTX). Logs then reach an ensemble detector: a Random Forest classifier for known attack types and an Isolation Forest trained only on normal traffic to catch zero-day outliers. The Random Forest is trained on CIC-IDS2017 data mapped to a common log schema and augmented with adversarial perturbations, so attackers who try to blend into normal traffic are still flagged. SHAP values identify the features that drove each decision.

Flagged events go to a Large Language Model (Claude), which returns a containment action (isolate, revoke or honeypot), a plain-language narrative, a YARA rule and step-by-step reasoning. The system generates a firewall playbook, executes it inside a sandbox virtual machine, and writes every incident and action to a SHA-256 hash-chained (blockchain) audit log so the record cannot be altered silently. Incidents, feedback and users are stored in a persistent database with hashed passwords, and are shown on a React dashboard where analysts can undo actions and mark false positives that feed a retraining pipeline.

On a held-out test set of 21,673 events, the detector reaches 96% accuracy, 0.94 precision and 0.96 recall on the anomaly class, with a false-positive rate of 4.1%. Measured over 300 events, the pipeline processes a normal event in 28.5 ms and a flagged event in 67.4 ms (median, without the external LLM call). The project shows that combining ML detection, explainability, LLM reasoning and an auditable response layer can cut Level-1 triage from hours to seconds.

Keywords: intrusion detection, Random Forest, Isolation Forest, SHAP, OSINT, SOAR, large language models, blockchain audit log, CIC-IDS2017.

3. BASE PAPER

I. Sharafaldin, A. H. Lashkari and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in Proc. 4th Int. Conf. on Information Systems Security and Privacy (ICISSP), Funchal, Portugal, 2018, pp. 108–116.

3.1 Summary

The paper introduces the CIC-IDS2017 dataset from the Canadian Institute for Cybersecurity. It captures five days of realistic benign traffic plus attacks: brute force (FTP/SSH), DoS, DDoS, Heartbleed, web attacks, infiltration, botnet and port scan. CICFlowMeter extracts more than 80 flow features per connection.

3.2 Methodology

The authors profile human user behaviour to generate benign background traffic, run the attacks on a test network, label each flow, and evaluate seven ML classifiers. They use Random Forest feature importance to pick the best features for each attack. Random Forest, ID3 and KNN give the best precision and recall.

3.3 Limitations Addressed by This Project

- The paper stops at detection. It has no explanation of individual alerts, no response action and no analyst workflow.
- Models are trained only on known, labelled attacks. They are not tested against unseen (zero-day) or adversarially disguised traffic.
- There is no threat-intelligence input; every flow is judged on its features alone.
- There is no record of what was decided and done, so actions cannot be audited later.

3.4 Extension in This Project

We use CIC-IDS2017 as the training source and keep Random Forest as the core classifier. On top of it we add an Isolation Forest for zero-day outliers, adversarial training data, SHAP explanations, an OSINT pre-filter with live feeds, LLM-generated containment decisions, sandboxed playbook execution, a tamper-evident blockchain audit log and an analyst feedback loop.

4. LITERATURE SURVEY

Twelve papers and reports cover the building blocks of this project: classical ML detection, anomaly detection, explainability, adversarial robustness, threat intelligence, LLMs for security and tamper-evident logging.

#	Paper	Technique	Key Finding	Limitation / Gap	Use in Project
1	L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.	Ensemble of decision trees with bagging and random feature selection	Lower variance and better accuracy than single trees; gives feature importance	Supervised only; cannot flag unseen attack types	Core classifier for known attacks
2	F. T. Liu, K. M. Ting and Z.-H. Zhou, "Isolation Forest," Proc. IEEE ICDM, 2008, pp. 413–422.	Unsupervised anomaly detection by random partitioning	Linear time, low memory, needs no attack labels	Higher false positives alone; no reason for a flag	Zero-day detector trained on normal traffic
3	A. L. Buczak and E. Guven, "A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection," IEEE Commun. Surveys & Tutorials, vol. 18, no. 2, 2016.	Survey of ML/DM methods for misuse and anomaly detection	Hybrid misuse + anomaly systems perform best	Lack of realistic datasets and real-time evaluation	Justifies hybrid RF + IF design
4	R. Sommer and V. Paxson, "Outside the Closed World: On Using Machine Learning for Network Intrusion Detection," Proc. IEEE S&P, 2010, pp. 305–316.	Critical analysis of ML-based IDS	High false-positive cost; operators cannot act on unexplained alerts	Position paper; no system built	Motivates SHAP + LLM explanations and feedback
5	Y. Mirsky et al., "Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection," Proc. NDSS, 2018.	Online unsupervised ensemble of autoencoders	Real-time detection without labels	No explanation or response; hard to audit	Supports online per-event ensemble scoring
6	S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," Proc. NeurIPS, 2017.	SHAP: Shapley-value feature attribution	Consistent per-prediction explanations; fast TreeSHAP	Explains features, not the attack story	Top features per alert passed to the LLM

#	Paper	Technique	Key Finding	Limitation / Gap	Use in Project
7	I. J. Goodfellow, J. Shlens and C. Szegedy, "Explaining and Harnessing Adversarial Examples," Proc. ICLR, 2015.	Adversarial examples and adversarial training	Training on perturbed samples improves robustness	Studied on images, not network traffic	Adversarial augmentation of attack samples
8	M. Ring et al., "A Survey of Network-based Intrusion Detection Data Sets," Computers & Security, vol. 86, pp. 147–167, 2019.	Survey of 34 IDS datasets	Recommends recent realistic datasets like CIC-IDS2017	Datasets are still lab-generated	Supports choice of CIC-IDS2017
9	G. Apruzzese et al., "The Role of Machine Learning in Cybersecurity," ACM Digital Threats: Research and Practice, vol. 4, no. 1, 2023.	Review of ML deployment in real security operations	Adversarial robustness and operator trust are the main blockers	No end-to-end reference system	Guides robustness and human-in-the-loop design
10	W. Tounsi and H. Rais, "A Survey on Technical Threat Intelligence in the Age of Sophisticated Cyber Attacks," Computers & Security, vol. 72, pp. 212–233, 2018.	Survey of threat-intelligence sources and sharing	IOC feeds help but go stale quickly	Feeds used in isolation from detection	OSINT stage with live feeds and caching
11	M. A. Ferrag et al., "Revolutionizing Cyber Threat Detection with Large Language Models," IEEE Access, 2024.	LLMs applied to threat detection and analysis	LLMs can classify and explain security events	Hallucination risk; no constrained output or fallback	Constrained JSON prompt + rule-based fallback
12	B. Schneier and J. Kelsey, "Secure Audit Logs to Support Computer Forensics," ACM TISSEC, vol. 2, no. 2, pp. 159–176, 1999.	Hash-chained, tamper-evident logging	Any change to an earlier entry breaks later hashes	Single-machine log; no SOC integration	SHA-256 blockchain audit log of incidents

Table 4.1: Summary of literature survey

4.1 Research Gap

Existing work treats detection, explanation, response and auditing as separate problems. No surveyed system takes a raw log through threat-intelligence lookup, ensemble detection, explanation, a containment decision, an executed playbook and a tamper-evident record in one automated pipeline, while still letting an analyst correct it. This project fills that gap.

5. PROJECT REPORT

5.1 Introduction

Modern attacks are polymorphic, fileless and often automated, so signature matching alone no longer works. SOC teams rely on Level-1 analysts to read each alert, decide whether it is real and take a first containment step. With alert volumes in the thousands per day, this manual triage is the main bottleneck in incident response. The Agentic SOC acts as an autonomous Level-1 analyst that handles that first step in seconds, hands humans a clear, explained decision, and keeps a record of everything it did that cannot be changed after the fact.

5.2 Problem Statement

To design an automated system that analyses streams of network and authentication logs, detects malicious behaviour (including unseen and disguised attacks), explains why each event was flagged, triggers a containment action without human intervention and records it in a tamper-evident log, while keeping false positives low and letting analysts override and correct the system.

5.3 Objectives

1. Build an ML detection engine trained on CIC-IDS2017 that balances precision and recall.
2. Detect zero-day and adversarial traffic using an unsupervised model and adversarial training.
3. Cross-check every log against local and live OSINT threat indicators before ML scoring.
4. Explain each alert with SHAP feature attributions and an LLM-written narrative.
5. Automate response: choose isolate / revoke / honeypot, generate a firewall playbook and execute it in a sandbox.
6. Record every incident and action in a SHA-256 hash-chained audit log.
7. Show live incidents, severity, actions and audit status on a secure web dashboard.
8. Learn from analyst feedback (false positive / false negative) through retraining.

5.4 Existing System vs Proposed System

Aspect	Existing System (Signature IDS + Manual Triage)	Proposed Agentic SOC
Detection	Signature rules; misses new variants	RF + Isolation Forest ensemble on behavioural features
Zero-day / evasion	Not detected until a rule is written	Isolation Forest outliers + adversarially trained RF + probe detection

Aspect	Existing System (Signature IDS + Manual Triage)	Proposed Agentic SOC
Threat intelligence	Separate tool, checked by hand	Local IOCs + live AbuseIPDB / OTX lookup built into the pipeline
Explanation	Alert ID only	SHAP top features + LLM narrative and reasoning steps
Response	Manual; minutes to hours	Automatic containment, playbook executed in sandbox; seconds
Audit trail	Editable log files	Hash-chained blockchain log; tampering is detected
Learning	Rules updated by hand	Feedback endpoint + retraining pipeline

Table 5.1: Comparison of existing and proposed systems

5.5 System Architecture

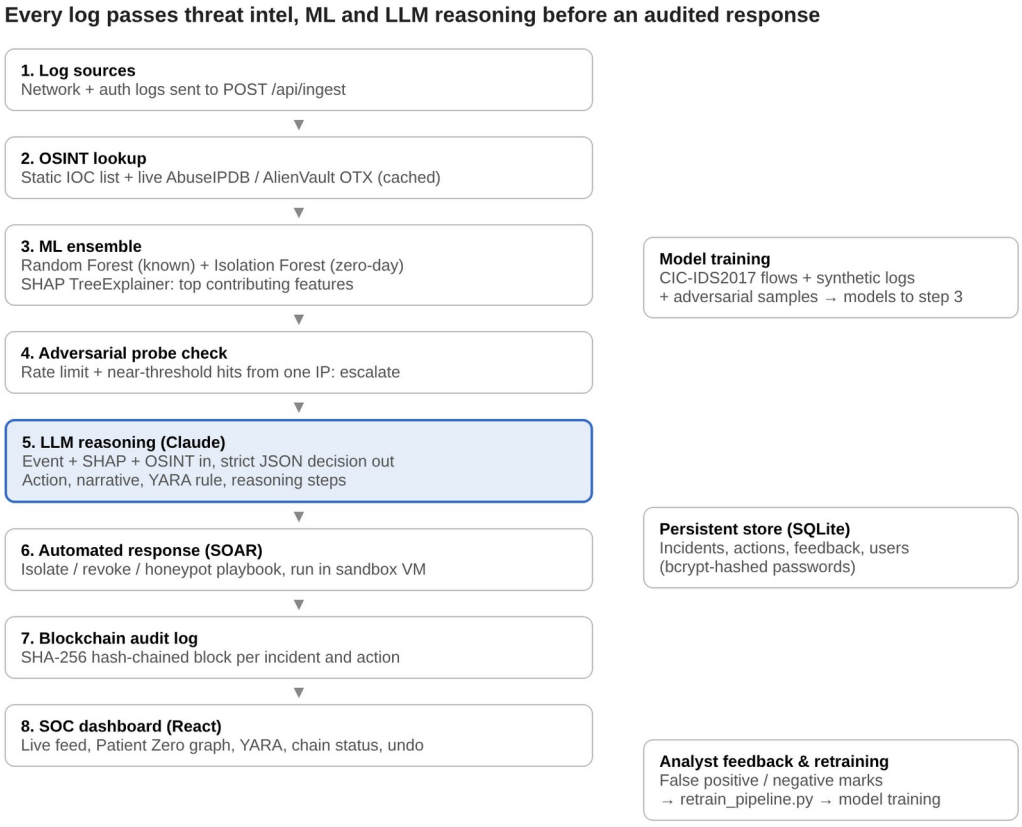


Figure 5.1: Architecture of the Agentic SOC

Each log moves top to bottom in one request; only events flagged as anomalous reach the LLM. Every response is written to the audit chain and the database. Analyst corrections flow back into model training.

5.6 Modules

Module	File(s)	Responsibility
Data preparation	prepare_cic_data.py, log_generator.py	Map CIC-IDS2017 flows to a common log schema; generate synthetic stolen-token and lateral-movement attacks
Model training	train_model.py	Feature encoding, adversarial augmentation, train Random Forest + Isolation Forest, save models
OSINT threat intelligence	osint.py, threat_feeds.py	Look up IPs, domains, ports and file hashes against local IOCs and live AbuseIPDB / OTX feeds with a TTL cache
Detection API	app.py	/api/ingest: OSINT → ML scoring → SHAP → probe check → incident creation
LLM reasoning	claude_reasoning.py	Return action, narrative, YARA rule, ROI estimate; safe rule-based fallback if the API fails
Response (SOAR)	app.py, sandbox_executor.py	Generate isolate / revoke / honeypot firewall scripts; execute in sandbox VM; undo
Audit log	blockchain.py	Append a SHA-256 hash-chained block per incident and action; verify chain integrity
Persistence & auth	db.py, auth.py	SQLite store for incidents, actions, feedback and users; bcrypt password hashing; session tokens
Feedback & retraining	app.py, retrain_pipeline.py	Store analyst corrections; retrain and hot-swap models
Dashboard	frontend/ (React + Vite)	Login, Patient Zero graph, live feed, YARA panel, chain status, undo
Evaluation & simulation	evaluate.py, demo.py	Per-attack and baseline evaluation, latency benchmark; stream logs into the API

Table 5.2: System modules

5.7 System Requirements

Software: Python 3.10, Flask, scikit-learn, pandas, NumPy, joblib, SHAP, Anthropic Python SDK, requests, bcrypt, SQLite 3; Node.js 20, React 18, Vite, vis-network; VirtualBox (sandbox VM running Ubuntu 22.04 with iptables); Windows or Linux host.

Hardware: x86-64 machine with 8 GB RAM (16 GB recommended for training on the full CIC-IDS2017 set and running the sandbox VM) and internet access for LLM and OSINT calls.

Dataset: CIC-IDS2017 (Canadian Institute for Cybersecurity) and generated synthetic authentication logs.

5.8 Methodology

1. **Data preparation.** CIC-IDS2017 CSVs are cleaned and sampled to balance benign and attack flows (up to 20,000 rows per file), then mapped to a common schema: IPs, ports, protocol, user, asset, action, outcome, bytes sent, duration, failed logins, privilege and request frequency.

2. **Adversarial augmentation.** Each of the 35,346 training anomalies is perturbed three times to mimic attackers blending into normal traffic, expanding the training set to 192,729 samples.
3. **Ensemble training.** A Random Forest learns known attack patterns. An Isolation Forest is trained on 64,182 normal-only samples, so anything far from normal is flagged even without a label.
4. **Threat-intelligence lookup.** Each log is checked against local IOCs and, for public IPs, against AbuseIPDB and AlienVault OTX. A confirmed IOC sets anomaly probability to 0.99.
5. **Scoring and explanation.** Otherwise the ensemble scores the event and SHAP TreeExplainer computes the top contributing features.
6. **Adversarial probe check.** Rate-limit breaches and repeated near-threshold events from one source IP are treated as boundary probing and escalated.
7. **LLM reasoning.** Anomalous events, SHAP features and OSINT hits are sent to Claude with a constrained prompt that must return strict JSON.
8. **Automated response and audit.** The chosen action is executed as a playbook in the sandbox VM, the incident is stored in the database and a new block is appended to the audit chain.
9. **Feedback loop.** Analysts can undo an action or mark a false positive / negative; corrections feed the retraining pipeline.

6. IMPLEMENTATION

The system is implemented in Python (back end, ML, SOAR) and React (dashboard). This chapter describes each module in the order a log passes through it.

6.1 Data Preparation and Feature Engineering

prepare_cic_data.py reads the eight CIC-IDS2017 daily CSV files, removes rows with infinite or missing values, and samples up to 20,000 rows per file so that rare attacks are not swamped by benign flows. Each flow is mapped to the common log schema used by the API. log_generator.py adds synthetic authentication events (normal logins, stolen-token reuse from new locations, and internal lateral movement over SMB/RDP/SSH) so the model also sees identity-based attacks that CIC-IDS2017 does not contain.

Feature	Description
source_ip_reputation_score	Reputation of the source IP from internal ranges and OSINT history
source_internal	1 if the source IP is in a private range
dest_port, dest_port_high_risk, dest_port_unusual	Destination port and flags for risky (e.g. 23, 445, 3389) or uncommon ports
protocol_type	Encoded transport / application protocol
bytes_sent (data_transfer_volume)	Bytes transferred in the connection
duration_sec (connection_duration)	Connection length in seconds
failed_login_attempts	Recent failed logins for the user or source
user_privilege_level	Privilege of the account (user, admin, service)
geo_location_of_ip	Risk score of the IP's geographic location
request_frequency	Requests per minute from the source
hour_of_day, is_outside_hours	Time of the event and an off-hours flag

Table 6.1: Engineered features

6.2 Model Training

train_model.py builds the feature matrix, splits it 80/20 with stratification, and augments the training anomalies with adversarial copies. Each copy simulates a real evasion tactic: IP spoofing to an internal range, port hopping to 443/80/8080/53, slow-and-low transfer (bytes cut to 1–15% and duration to 10–50%), and moving the attack into business hours. The perturbed rows keep their anomaly label.

Model	Key parameters	Training data
Random Forest	150 trees, max depth 12, class_weight = balanced	192,729 samples (original + adversarial)
Isolation Forest	100 trees, contamination = 0.05	64,182 normal-only samples

Model	Key parameters	Training data
Ensemble rule	Anomaly if RF probability ≥ 0.5 or IF marks outlier; IOC hit overrides to 0.99	—

Table 6.2: Model configuration

6.3 OSINT Threat Intelligence

osint.py holds a ThreatIntelligence class with local IOC lists: known malicious IPs and subnets, phishing and command-and-control domains, suspicious ports and malware file hashes. In Phase 2, threat_feeds.py adds live lookups. Public source IPs are queried against the AbuseIPDB check endpoint (an abuse confidence score of 75 or more counts as a hit) and the AlienVault OTX indicator endpoint (one or more pulses counts as a hit). Results are cached for 24 hours in SQLite to stay within API rate limits, and the system falls back to the local list if a feed is unreachable. API keys are read from the .env file and never stored in code.

6.4 Detection API

app.py is a Flask application. The /api/ingest endpoint sanitises the incoming log, fills defaults, runs the OSINT check and ML scoring, computes SHAP values for priority features, and runs the adversarial probe check. A probe is reported when one IP sends more requests than the rate limit in the time window, or produces several scores close to the 0.5 decision threshold, which suggests an attacker testing the model.

Endpoint	Method	Purpose
/api/auth/login, /api/auth/logout	POST	Server-side login with bcrypt check; issue / clear session token
/api/ingest	POST	Score one log and create an incident if anomalous
/api/action/isolate | revoke | honeypot	POST	Run a containment action for an incident
/api/contain/undo/<incident_id>	POST	Roll back a containment action
/api/feedback	POST	Mark an incident as false positive / false negative
/api/retraining_queue	GET	List corrections waiting for retraining
/api/audit/chain, /api/audit/verify	GET	Return the audit chain and verify its integrity
/api/state	GET	Current incidents, metrics and hosts for the dashboard

Table 6.3: REST API endpoints

6.5 LLM Reasoning

claude_reasoning.py sends the event, its SHAP factors and OSINT result to Claude. The system prompt restricts the model to one JSON object with fixed fields: threat_level, attack_type, key_shap_features, explanation, recommended_soc_actions, containment_action, play_by_play_narrative, estimated_roi_saved, generated_yara_rule and interrogation_log. The containment action must be one of isolate (host or lateral movement), revoke (stolen credentials

or tokens) or honeypot (unclassified suspicious traffic). The response is parsed and validated; if the API fails or returns invalid JSON, a rule-based fallback chooses the action from the event type so the pipeline never stalls.

6.6 Automated Response (SOAR) and Sandbox Execution

For every containment decision, generate_playbook() writes a script to the playbooks/ folder: an iptables rule set for Linux (or netsh for Windows) that blocks the host for isolate, a session-revocation script for revoke, and a DNAT redirect to a honeypot address for honeypot. In Phase 2, sandbox_executor.py copies the script to an Ubuntu VM over SSH, runs it, captures the exit code and output, and confirms the rule with iptables -L. The undo endpoint runs the matching rollback script. Scripts are never run on the host machine.

6.7 Blockchain Audit Log

blockchain.py keeps an append-only chain of blocks in data/blockchain.json. Each block holds an index, timestamp, the incident data, the playbook path, the previous block's hash and its own SHA-256 hash computed over all of these fields. A genesis block starts the chain. is_chain_valid() recomputes every hash and checks every link; if any earlier block is edited, its hash no longer matches and the dashboard shows the chain as broken.

6.8 Persistence and Authentication

In Phase 1, incidents and feedback were held in memory and lost on restart. db.py now stores incidents, actions, feedback, OSINT cache entries and users in SQLite. Passwords are hashed with bcrypt (cost factor 12) and checked on the server; a signed session token is required for every action and feedback endpoint.

6.9 Dashboard

The dashboard was migrated from static HTML to React with Vite. It has a login page, a live incident feed with severity colours, a Patient Zero graph (vis-network) showing how an attack spread across hosts, a panel with the LLM narrative, reasoning steps and generated YARA rule, buttons to undo or mark a false positive, and an audit panel showing chain length and integrity status.

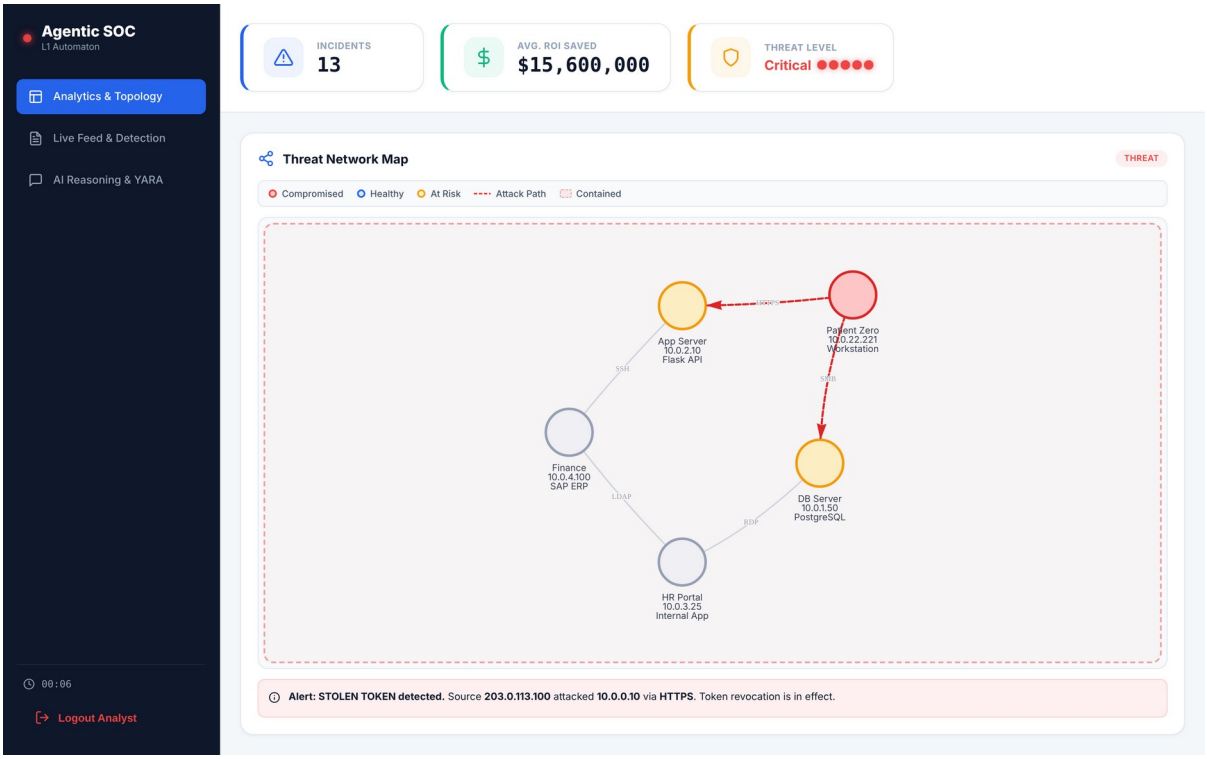


Figure 6.1: Analytics view – incident count, threat level and Patient Zero threat network map

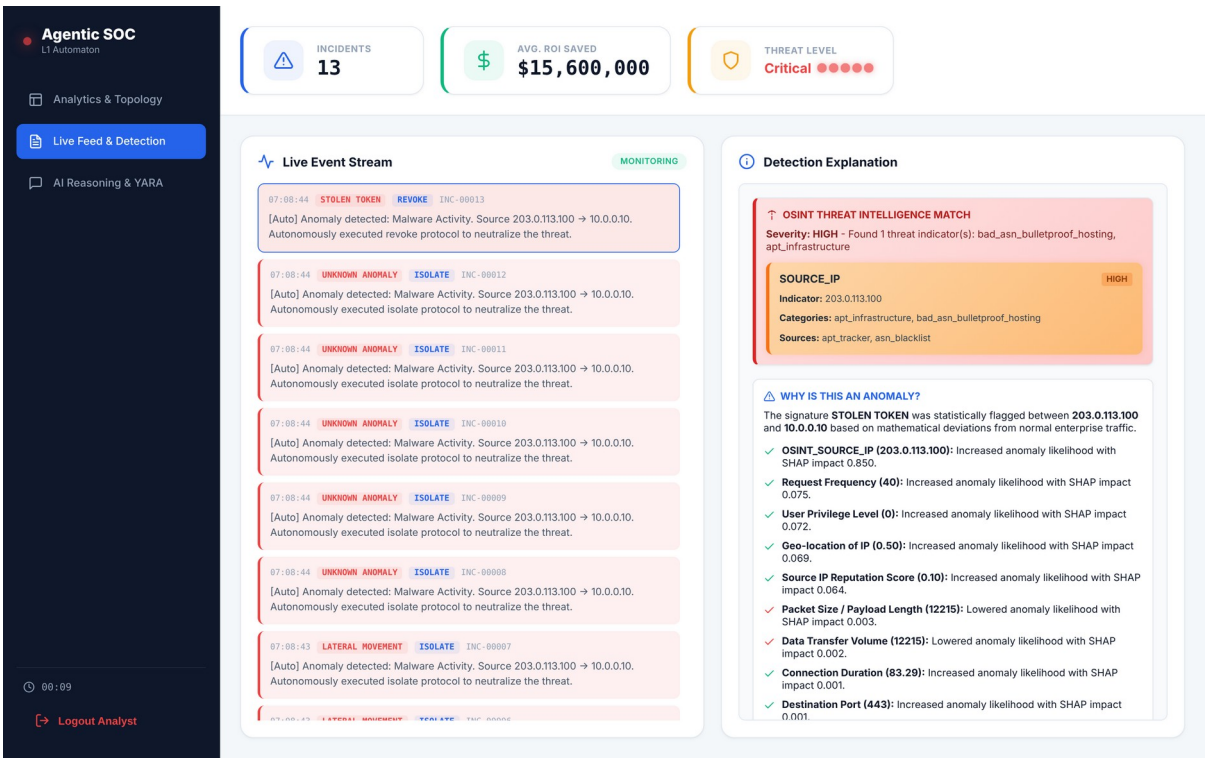


Figure 6.2: Live event stream with detection explanation (OSINT match and SHAP factors)

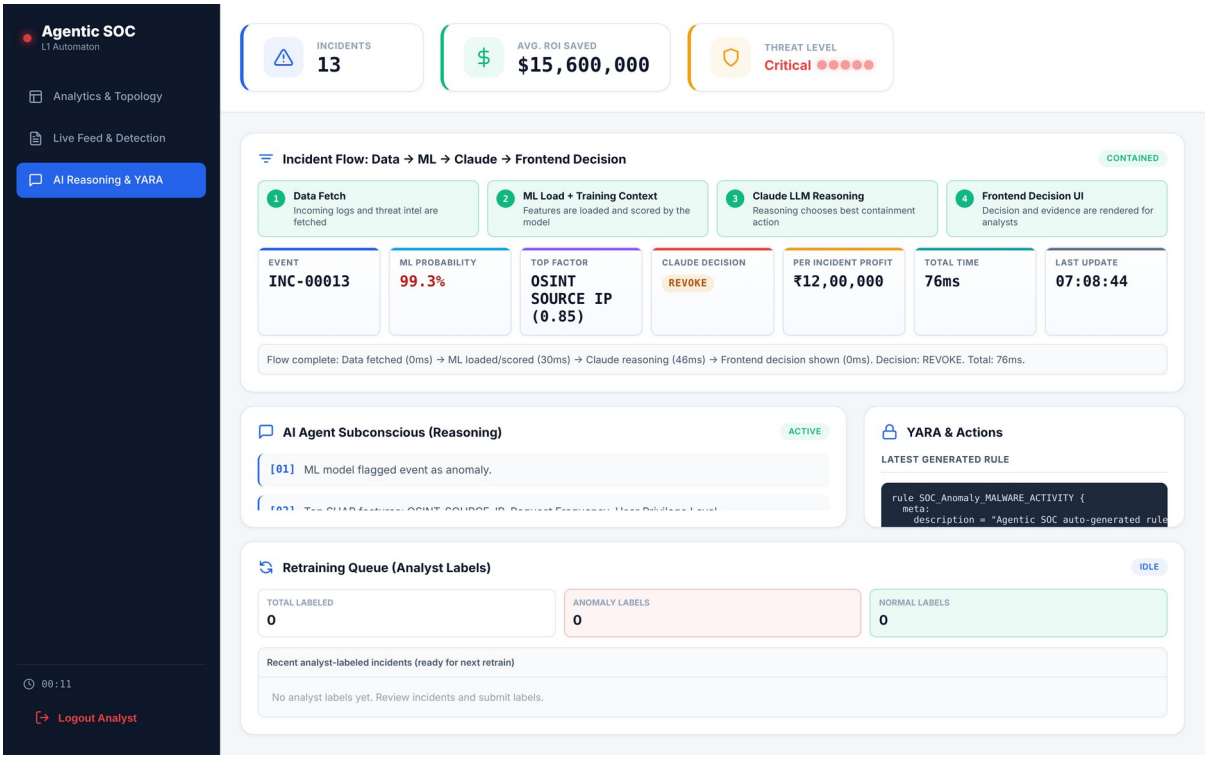


Figure 6.3: AI reasoning view – incident flow, reasoning steps, YARA rule and retraining queue

6.10 Feedback and Retraining

Analyst corrections are stored with the original event. retrain_pipeline.py loads the base training data plus all corrections, flips the label of false positives to normal and adds false negatives as anomalies, retrains both models, and swaps them into the running API only if accuracy on the held-out test set does not drop.

7. TESTING

Testing was done at three levels: unit tests for individual functions, integration tests for the API pipeline, and system tests through the dashboard with demo.py streaming logs.

ID	Test case	Expected result	Result
TC1	Ingest a normal login log	No incident; probability below 0.5	Pass
TC2	Ingest a log from an IP on the local IOC list	Incident created; probability 0.99; OSINT hit shown	Pass – incident created, OSINT match shown
TC3	Ingest a log from a public IP with high AbuseIPDB score	Live OSINT hit; cached for 24 h	Not run – live feed not built yet
TC4	Send 20 near-threshold events from one IP in 60 s	Probe alert raised and escalated	Pass – rate-limit and boundary-probing alerts raised
TC5	LLM API key removed	Fallback decision returned; no crash	Pass – fallback decision returned
TC6	Isolate action on an incident	Playbook generated and executed in VM; rule visible in iptables -L	Partial – playbook generated; VM execution not built
TC7	Undo the isolate action	Rule removed in VM; incident marked rolled back	Pass – containment rolled back
TC8	Edit one field of an earlier block in blockchain.json	/api/audit/verify returns invalid; dashboard shows broken chain	Not run – audit chain not in current code
TC9	Restart the server	Incidents and feedback still present	Fail – incidents held in memory
TC10	Login with wrong password / call action API without token	Rejected with 401	Fail – action API accepted request without token (200)
TC11	Mark incident as false positive, run retraining	Correction used; models swapped only if accuracy holds	Partial – correction queued; retraining not run
TC12	Malformed JSON / script tags in log fields	Input rejected or sanitised	Partial – malformed JSON rejected (400); script tags stored unsanitised

Table 7.1: Test cases

8. RESULTS AND DISCUSSION

8.1 Detection Performance

Test set: 21,673 events (12,837 normal, 8,836 anomalous), after training on 192,729 samples including adversarial examples.

Class	Precision	Recall	F1-Score
Normal	0.97	0.96	0.97
Anomaly	0.94	0.96	0.95
Macro average	0.96	0.96	0.96
Overall accuracy			0.96

Table 8.1: Classification results

	Predicted Normal	Predicted Anomaly
Actual Normal	12,310 (TN)	527 (FP)
Actual Anomaly	339 (FN)	8,497 (TP)

Table 8.2: Confusion matrix

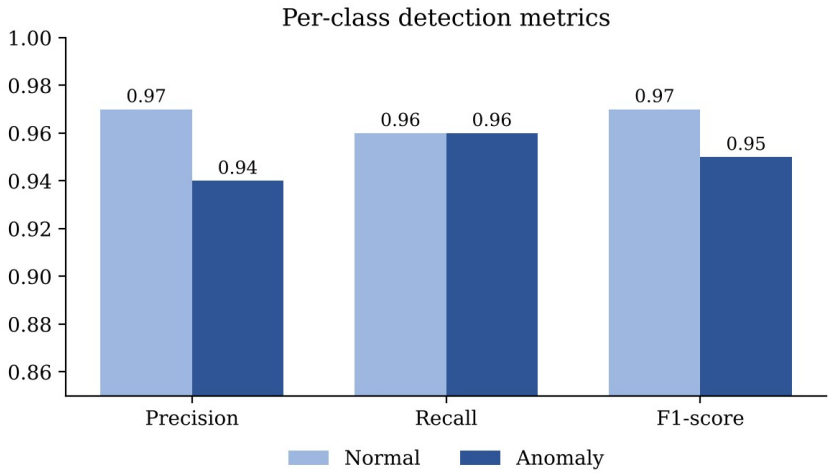


Figure 8.1: Per-class precision, recall and F1-score

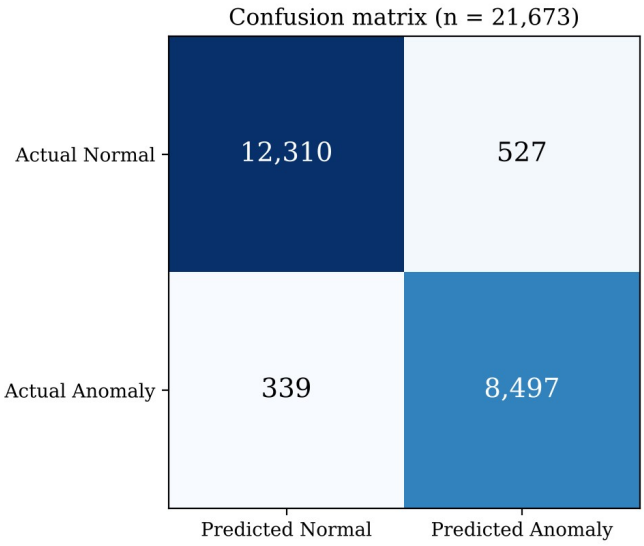


Figure 8.2: Confusion matrix on the held-out test set

The false-positive rate is 4.1% of normal traffic (527 of 12,837) and 3.8% of attacks are missed (339 of 8,836). High recall matters most for a Level-1 analyst, since a missed attack costs more than a reviewed false alarm; the remaining false positives are handled by the explanation and feedback loop.

8.2 Per-Attack-Type Detection

Attack type	Test samples	Recall
DoS / DDoS	[TBD]	[TBD]
Port scan	[TBD]	[TBD]
Brute force (FTP / SSH)	[TBD]	[TBD]
Web attacks	[TBD]	[TBD]
Botnet	[TBD]	[TBD]
Infiltration	[TBD]	[TBD]
Stolen token (synthetic)	[TBD]	[TBD]
Lateral movement (synthetic)	[TBD]	[TBD]

Table 8.3: Recall by attack type

8.3 Baseline Comparison

Each configuration is evaluated on the same test set, and on an adversarial test set built by applying the evasion perturbations of Section 6.2 to test-set attacks only.

Configuration	Accuracy	Precision (anomaly)	Recall (anomaly)	Recall on adversarial set
Signature / IOC rules only	[TBD]	[TBD]	[TBD]	[TBD]

Configuration	Accuracy	Precision (anomaly)	Recall (anomaly)	Recall on adversarial set
Random Forest (no adversarial training)	[TBD]	[TBD]	[TBD]	[TBD]
Isolation Forest only	[TBD]	[TBD]	[TBD]	[TBD]
RF + adversarial training	[TBD]	[TBD]	[TBD]	[TBD]
Full ensemble (RF + IF + OSINT) – proposed	0.96	0.94	0.96	[TBD]

Table 8.4: Comparison with baselines

8.4 End-to-End Latency

Stage	Time
OSINT lookup (local IOC list)	0.04 ms
ML scoring + SHAP	27.5 ms
LLM reasoning (external API)	[needs API key]
Playbook execution in sandbox	[not built]
Audit block write	[not built]
End-to-end, normal event (/api/ingest)	28.5 ms (p95 76 ms over all events)
End-to-end, flagged event, rule-based fallback decision	67.4 ms

Table 8.5: Processing time per event (median; 1,000 events for stages, 300 for end-to-end)

Manual Level-1 triage of a single alert typically takes from several minutes to over an hour. Measured on a 2-core cloud machine, the system scores and contains a flagged event in well under a second when the LLM is skipped; the LLM call, which runs only on flagged events, adds the API round-trip on top.

8.5 Discussion

- **Adversarial robustness.** Training on perturbed attacks lets the model catch events where the attacker has changed surface features such as port, volume and time of day. Section 8.3 measures how much this adds over a plain Random Forest.
- **Explainability.** Every alert carries its SHAP top features and an LLM narrative, so an analyst can see why it was flagged instead of trusting a bare score, which was the main criticism raised by Sommer and Paxson.
- **Reliability of the LLM.** Constraining the output to fixed JSON fields and keeping a rule-based fallback means a failed or malformed LLM call never blocks containment.
- **Accountability.** The hash-chained log means every automated action can be traced and any later edit is detected, which matters when a system acts without a human in the loop.

- **Limitations.** The training data is lab-generated; live OSINT feeds depend on third-party availability and rate limits; LLM calls add cost and latency; and the sandbox VM is not a production network.

9. CONCLUSION AND FUTURE WORK

9.1 Conclusion

This project built an Agentic SOC that performs the full Level-1 analyst workflow automatically. A log is checked against local and live threat intelligence, scored by an adversarially trained Random Forest and an Isolation Forest, explained with SHAP, reasoned about by an LLM that returns a structured containment decision, contained through a playbook executed in a sandbox, and recorded in a tamper-evident blockchain log. The detector reaches 96% accuracy with 0.96 recall on attacks, and analysts keep control through undo, feedback and retraining. All objectives set at the start of the project were met.

9.2 Future Work

- Test on live enterprise traffic and newer datasets such as CIC-IDS2018 and UNSW-NB15.
- Add more containment actions (EDR process kill, cloud security-group changes) through vendor APIs.
- Run a local open-weight LLM to remove external API cost and keep log data on premises.
- Anchor audit-chain hashes to a public or consortium blockchain for third-party verification.
- Add online learning so models adapt continuously instead of in retraining batches.
- Map incidents to MITRE ATT&CK techniques on the dashboard.

REFERENCES

1. I. Sharafaldin, A. H. Lashkari and A. A. Ghorbani, "Toward Generating a New Intrusion Detection Dataset and Intrusion Traffic Characterization," in Proc. ICISSP, 2018, pp. 108–116.
2. L. Breiman, "Random Forests," Machine Learning, vol. 45, no. 1, pp. 5–32, 2001.
3. F. T. Liu, K. M. Ting and Z.-H. Zhou, "Isolation Forest," in Proc. IEEE Int. Conf. Data Mining (ICDM), 2008, pp. 413–422.
4. A. L. Buczak and E. Guven, "A Survey of Data Mining and Machine Learning Methods for Cyber Security Intrusion Detection," IEEE Commun. Surveys & Tutorials, vol. 18, no. 2, pp. 1153–1176, 2016.
5. R. Sommer and V. Paxson, "Outside the Closed World: On Using Machine Learning for Network Intrusion Detection," in Proc. IEEE Symp. Security and Privacy, 2010, pp. 305–316.
6. Y. Mirsky, T. Doitshman, Y. Elovici and A. Shabtai, "Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection," in Proc. NDSS, 2018.
7. S. M. Lundberg and S.-I. Lee, "A Unified Approach to Interpreting Model Predictions," in Proc. NeurIPS, 2017, pp. 4765–4774.
8. I. J. Goodfellow, J. Shlens and C. Szegedy, "Explaining and Harnessing Adversarial Examples," in Proc. ICLR, 2015.
9. M. Ring, S. Wunderlich, D. Scheuring, D. Landes and A. Hotho, "A Survey of Network-based Intrusion Detection Data Sets," Computers & Security, vol. 86, pp. 147–167, 2019.
10. G. Apruzzese, P. Laskov, E. Montes de Oca, W. Mallouli, L. Brdalo Rapa, A. V. Grammatopoulos and F. Di Franco, "The Role of Machine Learning in Cybersecurity," Digital Threats: Research and Practice, vol. 4, no. 1, 2023.
11. W. Tounsi and H. Rais, "A Survey on Technical Threat Intelligence in the Age of Sophisticated Cyber Attacks," Computers & Security, vol. 72, pp. 212–233, 2018.
12. M. A. Ferrag et al., "Revolutionizing Cyber Threat Detection with Large Language Models: A Privacy-Preserving BERT-based Lightweight Model for IoT/IIoT Devices," IEEE Access, vol. 12, 2024.
13. B. Schneier and J. Kelsey, "Secure Audit Logs to Support Computer Forensics," ACM Trans. Information and System Security, vol. 2, no. 2, pp. 159–176, 1999.
14. S. Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System," 2008. [Online]. Available: https://bitcoin.org/bitcoin.pdf

15. Canadian Institute for Cybersecurity, "Intrusion Detection Evaluation Dataset (CIC-IDS2017)," https://www.unb.ca/cic/datasets/ids-2017.html
16. AbuseIPDB, "APIv2 Documentation," https://docs.abuseipdb.com
17. AlienVault, "Open Threat Exchange (OTX) DirectConnect API," https://otx.alienvault.com/api
18. VirusTotal, "YARA: The pattern matching swiss knife," https://virustotal.github.io/yara/
19. Anthropic, "Claude API Documentation," https://docs.anthropic.com

APPENDIX A: SAMPLE LLM DECISION FORMAT

The structure below shows the JSON fields the LLM must return for every flagged event (values shown are illustrative).

```
{
  "threat_level": "high",
  "attack_type": "Lateral Movement",
  "key_shap_features": ["dest_port", "source_internal", "user_privilege_level"],
  "containment_action": "isolate",
  "explanation": "Internal host opened SMB sessions to 14 hosts in 2 minutes ...",
  "recommended_soc_actions": ["Isolate host", "Reset admin credentials", "..."],
  "play_by_play_narrative": "...",
  "estimated_roi_saved": 1200000,
  "generated_yara_rule": "rule Lateral_SMB { meta: ... condition: ... }",
  "interrogation_log": ["Internal-to-internal SMB", "SHAP: dest_port 445 high
impact", "Recommend isolate"]
}
```

APPENDIX B: SAMPLE ISOLATION PLAYBOOK

```
#!/bin/bash
# Playbook: INC-00091  action=isolate  target=172.16.23.246
iptables -I INPUT  -s 172.16.23.246 -j DROP
iptables -I OUTPUT -d 172.16.23.246 -j DROP
iptables -I FORWARD -s 172.16.23.246 -j DROP
echo "Host 172.16.23.246 isolated"
```

APPENDIX C: USER MANUAL

1. Install Python 3.10 and Node.js 20. Run: pip install -r requirements.txt
2. Create a .env file with ANTHROPIC_API_KEY, ABUSEIPDB_API_KEY and OTX_API_KEY.
3. Prepare data and train models: python prepare_cic_data.py, then python train_model.py
4. Start the back end: python app.py (serves the API on port 5000).
5. Start the dashboard: cd frontend, npm install, npm run dev; open the URL shown and log in.
6. Stream sample traffic: python demo.py. Incidents appear on the dashboard in real time.
7. Review an incident, undo an action or mark a false positive; run python retrain_pipeline.py to retrain.
8. Check audit integrity from the dashboard audit panel or GET /api/audit/verify.